



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV TELEKOMUNIKACÍ

DEPARTMENT OF TELECOMMUNICATIONS

MULTIPLATFORMNÍ APLIKACE PRO HROMADNOU SPRÁVU ZAŘÍZENÍ MIKROTIK ROUTEROS

MULTI-PLATFORM APPLICATION FOR BULK MANAGEMENT OF MIKROTIK ROUTEROS DEVICES

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Jiří Kozárek

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. Ondřej Krajsa, Ph.D.

BRNO 2026

Bakalářská práce

bakalářský studijní program **Telekomunikační a informační systémy**

Ústav telekomunikací

Student: Jiří Kozárek

ID: 256766

Ročník: 3

Akademický rok: 2025/26

NÁZEV TÉMATU:

Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS

POKYNY PRO VYPRACOVÁNÍ:

Navrhněte a implementujte multiplatformní aplikaci (Windows, Linux) v Pythonu pro správu zařízení s RouterOS prostřednictvím oficiálního API (šifrované připojení TLS na portu 8729). Aplikace musí umožnit kompletní správu jednotlivých zařízení, bezpečné uložení přístupových údajů, jednotlivé i hromadné provádění konfiguračních změn (např. přes šablony), zálohu a obnovu konfigurace a jednoduchou správu logů. Výstupem práce bude plně funkční aplikace, zdrojové kódy a uživatelská a instalační příručka.

DOPORUČENÁ LITERATURA:

Podle pokynů vedoucího práce.

Termín zadání: 9.2.2026

Termín odevzdání: 2.6.2026

Vedoucí práce: Ing. Ondřej Krajsa, Ph.D.

prof. Ing. Jiří Mišurec, CSc.
předseda rady studijního programu

UPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

ABSTRAKT

Bakalářská práce má za cíl vytvořit multiplatformní aplikaci v programovacím jazyce Python, jež bude schopna provádět hromadné konfigurace aktivních síťových prvků, na kterých běží operační systém MikroTik RouterOS. Konfigurace bude probíhat z grafického rozhraní pomocí systémového rozhraní API na portu 8729, konkrétně jeho šifrované verze zabezpečené pomocí TLS. Celá aplikace je dostupná v repozitáři na platformě GitHub.

KLÍČOVÁ SLOVA

Python, MikroTik, RouterOS, hromadná konfigurace

ABSTRACT

The goal of this bachelor's thesis is creation of cross-platform application written in Python programming language that will be capable of mass configuration of active network devices running MikroTik RouterOS operating system. The configuration will be done via graphical user interface using system API interface on port 8729, specifically its encrypted version secured by TLS. The entire app is available in a repository on GitHub platform.

KEYWORDS

Python, MikroTik, RouterOS, mass configuration

KOZÁREK, Jiří. *Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav telekomunikací, 2026. Vedoucí práce: Ing. Ondřej Krajsa, Ph.D.

Prohlášení autora o původnosti díla

Jméno a příjmení autora:	Jiří Kozárek
VUT ID autora:	256766
Typ práce:	Bakalářská práce
Akademický rok:	2025/26
Téma závěrečné práce:	Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS

Prohlašuji, že svou závěrečnou práci jsem vypracoval samostatně pod vedením vedoucí/ho závěrečné práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury na konci práce. V souvislosti s vytvořením závěrečné práce jsem neporušil zásady a doporučení VUT k využívání generativní umělé inteligence.

Jako autor uvedené závěrečné práce dále prohlašuji, že v souvislosti s vytvořením této závěrečné práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a/nebo majetkových a jsem si plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

Brno
.....
podpis autora*

*Autor podepisuje pouze v tištěné verzi.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu bakalářské práce panu Ing. Ondřeji Krajsovi, Ph.D. za vedení, konzultace a podnětné návrhy k práci. Dále bych chtěl také poděkovat panu Bc. Jakubu Raimrovi za rady při psaní této práce a panu Martinu Smejkalovi za věcné rady v oblasti zabezpečení dat, bezpečnosti a programování.

Obsah

Úvod	10
1 Síťová zařízení	11
1.1 MikroTik	11
2 RouterOS	12
2.1 Konfigurace	12
2.2 winbox/webfig	12
2.3 API	14
3 Návrh aplikace	15
3.1 Databáze	15
3.1.1 Možné realizace databáze	15
3.1.2 Ukládání hesel	16
3.2 Grafické rozhraní	17
3.3 Pohledy	17
3.4 Bezpečnost aplikace	18
3.5 Hromadné operace	19
4 Implementace	20
4.1 Komunikace se systémem RouterOS pomocí API	20
Závěr	22
Literatura	23
Seznam symbolů a zkratk	25
Seznam příloh	26

Seznam obrázků

2.1	Některé z možností konfigurace ROS	12
2.2	Ukázka grafického rozhraní WinBox	13
2.3	Ukázka webového rozhraní WebFig	13
3.1	Jednoduchý ilustrační diagram interakce jednotlivých částí a aplikací mezi sebou	15
3.2	Grafické rozhraní konfiguračního okna (vývojová verze)	17
3.3	Diagram návrhu realizace hromadné konfigurace	19

Seznam výpisů

2.1	Ukázka komunikace pomocí api s využitím ukázkového skriptu [11], zkráceno	14
4.1	Ukázka přihlášení při komunikaci s rozhraním API	20

Úvod

Bakalářská práce se zaměřuje na návrh a realizaci multiplatformní aplikace pro hromadnou správu zařízení s operačním systémem RouterOS společnosti MikroTik. Pro komunikaci se zařízeními má aplikace využívat zabezpečenou komunikaci s rozhraním API na portu 8729.

První kapitola zběžně uvádí čtenáře do tématu síťových prvků a společnosti MikroTik.

Druhá kapitola pojednává o systému RouterOS společnosti MikroTik. Uvádí vybrané způsoby jeho konfigurace se zaměřením na aplikaci WinBox a rozhraní API. Třetí kapitola řeší problematiku databáze. Probírá její strukturu i možnosti realizace. Dále také pojednává o způsobu uložení přihlašovacích údajů.

Čtvrtá kapitola se zaměřuje na grafické rozhraní, jeho výběru a celkové fungování a vzhled aplikace.

Pátá kapitola pojednává o systémem pohledů, který se v systému RouterOS běžně nenachází a který se tato aplikace snaží implementovat.

Šestá kapitola popisuje důvody vynechání vývoje této části aplikace.

Sedmá kapitola pojednává o skutečné realizaci aplikace, jejím ovládání a chování. Tato část se věnuje čistě realizaci aplikace.

1 Síťová zařízení

Dnešní svět, silně propojený celosvětovou počítačovou sítí Internet, si bez její existence již nedokážeme představit. Díky této síti jsme schopni docílit téměř real-time komunikace, a to z kteréhokoli místa na světě na jiné. Vznik takovéto sítě je podmíněn vysokým stupněm standardizace [1]. Síť, skládající se z pasivních a aktivních částí, vyžaduje správu.

1.1 MikroTik

Na světě existuje nespočet firem, malých či velkých, které se zabývají vývojem nebo výrobou speciálního softwaru či hardwaru pro síťové aplikace. Namátkou lze vybrat firmy známé, jejichž jména se stala v oboru ikonickými, jako jsou společnosti CISCO Systems [2], HP (Hewlett-Packard) [3] a její odnož Enterprise [4], či Arista [5]. Z řady menších, ale stále dosti významných firem, stojí za zmínku firma Ubiquiti [6], která se snaží o zjednodušení konfigurace a správy aktivních síťových prvků. Zaměřením této práce je však litevská firma SIA Mikrotīks, známá jako MikroTik [7]. Jedná se o výrobce kompletních síťových řešení v oblasti aktivních prvků ve smyslu výroby vlastního síťového hardwaru [8] a především síťového softwaru RouterOS [9].

2 RouterOS

Síťový operační systém ROS společnosti MikroTik je komunitou vysoce oblíbený a rozšířený napříč spektrem síťových zařízení. Jeho distribuce probíhá společně s hardwarovými produkty. Díky oblibě komunity se také stal samostatným produktem společnosti. Lze jej pořídit samostatně a provozovat přímo na strojích s architekturou x86 či využít verzi CHR a provozovat tento systém jako virtuální stroj. Tímto operačním systémem je nabízena konfigurace nepřeberného množství protokolů převážně na prvních třech vrstvách ISO/OSI modelu [10].

2.1 Konfigurace

Systémem ROS je nabízeno velké množství způsobů konfigurace. K navigaci je použita stromová struktura příkazů, ve které je třeba se pro podrobnější správu vydat po dané větvi dál od konfiguračního kmene až k samotným možným argumentům příkazů. Na následujícím obrázku je zobrazen seznam možností přístupů konfigurace a port, na kterém služba aktuálně naslouchá. Stojí za zmínku, že všechny zobrazené služby běží na standardních (výchozích) portech.

Name	Port
api	8728
api-ssl	8729
ftp	21
ssh	22
telnet	23
winbox	8291
www	80
www-ssl	443

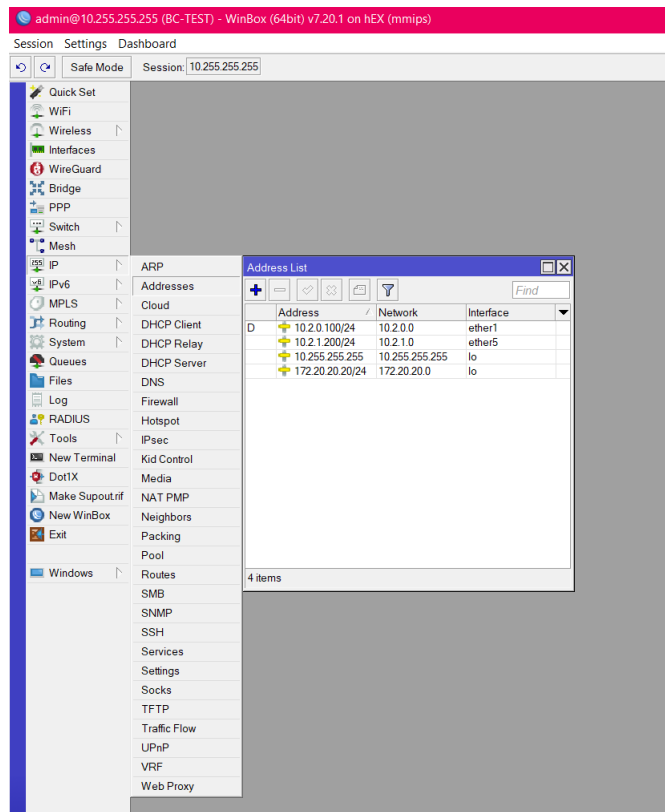
Obr. 2.1: Některé z možností konfigurace ROS

Pro účely této práce jsou však jiné způsoby konfigurace nežli *winbox/webfig* a *api* bezvýznamné, proto jim již nebude v této práci věnována větší pozornost.

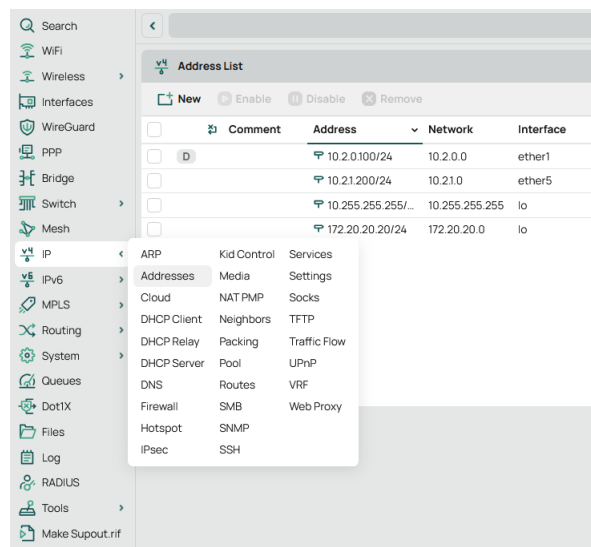
2.2 winbox/webfig

WinBox je grafický proprietární konfigurační software pro správu zařízení se systémem Mikrotik RouterOS. Je založen na systému oken s konfiguračními podokny, pomocí kterých se správa provádí. Lze konstatovat, že se jedná o grafické zobrazení stromové hierarchie příkazů dostupných v konzoli. Stejně tak lze konstatovat, že konfigurační rozhraní webfig plní funkci winboxu. Jedná se však o webovou aplikaci vzhledově a do jisté míry i funkčně podobnou aplikaci winbox. Její největší výhoda

spočívá v možnosti konfigurace prakticky z každého zařízení, na kterém lze spustit webový prohlížeč.



Obr. 2.2: Ukázka grafického rozhraní WinBox



Obr. 2.3: Ukázka webového rozhraní WebFig

2.3 API

Jedná se o přímé napojení využívající komunikaci na úrovni bytů, kterou je třeba dekodovat. Samotná užitečná komunikace pak probíhá ve formátu příkazového slova společně s jeho argumenty, následuje odpověď daného zařízení ve formě výčtu ve formátu parametr – hodnota.

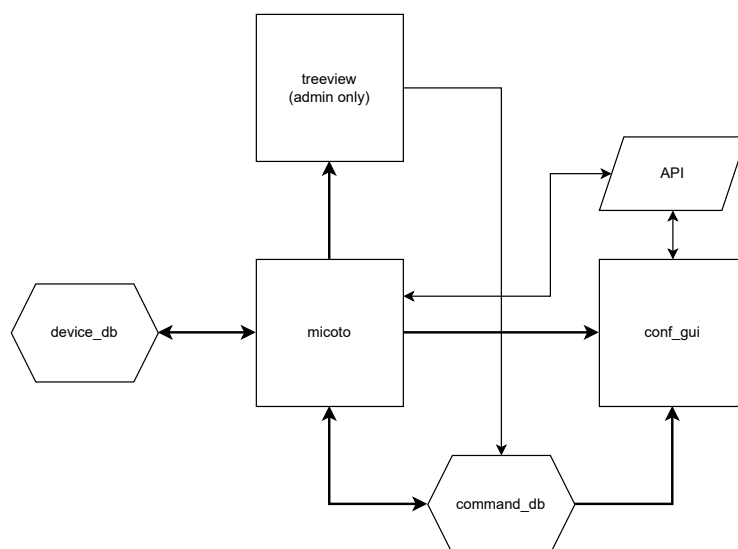
Výpis 2.1: Ukázka komunikace pomocí api s využitím ukázkového skriptu [11], zkráceno

```
<<< /ip/address/print
<<<
>>> !re
>>> =.id=*2
>>> =address=10.255.255.255/32
>>> =network=10.255.255.255
>>> =interface=lo
>>> =actual-interface=lo
>>> =invalid=false
>>> =dynamic=false
>>> =slave=false
>>> =disabled=false
```

Ukázka nevyužívá všechna možná slova a parametry, které lze použít při komunikaci. Pro ukázkou je tento jednoduchý příkaz dostatečný. Dále je také v práci uvažována komunikace s api rozhraním v zabezpečené formě využívající TLS - v ukázce pod názvem *api-ssl*.

3 Návrh aplikace

Aplikace je rozdělena na několik samostatně funkčních bloků, a to samotnou aplikaci *micoto* v uživatelském či administrátorském režimu, aplikaci *treeview* sloužící ke grafické reprezentaci stromu příkazů systému RouterOS a jejich výběru a uložení do databáze a konfigurační aplikaci *conf_gui* sloužící k realizaci samotných konfiguračních změn.



Obr. 3.1: Jednoduchý ilustrační diagram interakce jednotlivých částí a aplikací mezi sebou

3.1 Databáze

Jedním ze základních úkolů této práce pro autora byl návrh databáze a systému ukládání hesel. Struktura databáze má dalekosáhlé důsledky pro návrh celé architektury aplikace a jejím budoucím možnostem.

3.1.1 Možné realizace databáze

Autor vybíral ze základních dvou přístupů pro práci s databází. Jedním z nich je přístup pomocí serveru. V tomto modelu je třeba serverová část, spuštěná lokálně či na vzdáleném serveru, ke které se uživatel připojí a využívá její služby pomocí síťového stacku operačního systému. Má však velké úskalí, díky kterému je dle autora nevhodná pro tuto aplikaci.

Při lokálně spuštěné serverové instanci je nejprve nutno tuto instanci na daném počítači nainstalovat a spustit. Takové řešení sice je možné, ale autor má za to, že takové řešení přidává zbytečnou komplexitu projektu.

Při použití vzdáleného serveru je předpokládána funkčnost síťového spojení na daný server a jeho funkčnost. Z logiky vyplývá, že by zde mohl nastat zásadní problém, a to v momentu výpadku. Nastala by situace, kdy by síťové prvky, nutné pro spojení s databázovým serverem, nemohly být nakonfigurovány, jelikož přístupové údaje k těmto prvkům by byly uloženy na v tu chvíli nedostupném serveru. Z výše uvedeného vyplývají zásadní možné chyby a problémy dané realizace databáze, proto autor vybral alternativní metodu uvedenou níže.

SQLite

Díky předešlým úvahám a vlastnostem uvedených systémů autor využil v aplikaci SQLite [12], konkrétně implementaci tohoto systému knihovnou sqlite3 [13]. Jedná se o lightweight knihovnu obsaženou přímo v pythonu, není tedy třeba její doinstalace. Tento systém nevyužívá spojení na databázový server, nýbrž lokální databázový soubor. Aplikace tedy bude fungovat i v případě kompletního odpojení od sítě či jejím výpadku. Je třeba zmínit hlavní nevýhodu tohoto přístupu, a tou je nutnost databázové soubory distribuovat či synchronizovat. Dále také fakt, že nemůže být přistupováno k databázovému souboru vícero uživateli najednou. Při využívání databázového souboru je soubor uzamčen pro čtení i zápis ostatním uživatelům. Potenciální problémy s přístupem k databázovému souboru však nejsou pro tuto aplikaci relevantní, jelikož aplikaci může využívat v daném okamžiku na jednom počítači pouze jeden uživatel. Otázka distribuce a synchronizace bude popsána dále. Proto má autor za to, že řešení pomocí systému SQLite, používající lokální databázové soubory, bude nejvhodnějším řešením.

3.1.2 Ukládání hesel

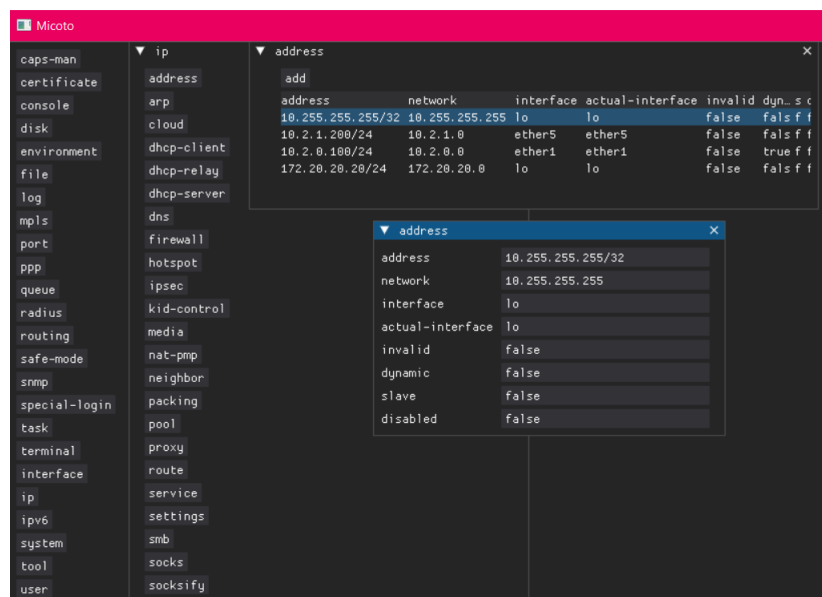
Ačkoli je autorovi známo, že ukládání hesel jako takových je naprosto nevhodné a silně nedoporučené, v tomto případě není zbylí. Systém ROS nepodporuje pro přístup pomocí api jinou autentizační metodu nežli přihlášení uživatelským jménem a heslem. Jelikož je jeden pohled aplikace zamýšlen spíše uživatelsky, viz dále, je nutné vytvořit jednoduchý systém, kdy po zadání jednoho hesla budou zpřístupněna všechna zařízení. Protože nastavení stejného hesla do vícero zařízení není z bezpečnostního hlediska lepším řešením, zvolil autor řešení využívající databázi se zašifrovanými přístupovými hesly.

Prvně bylo autorem zamýšleno využít systém symetrického šifrování s využitím

knihovny cryptography, a to konkrétně již připraveným systémem Fernet [14]. Tento systém je jednoduchý na implementaci, avšak po konzultaci s vedoucím práce bylo rozhodnuto pro jeho nevyužití, jelikož pro šifrování je využíván AES-128-CBC. Ačkoli je AES-128 dnes stále považován za bezpečný a splňuje minimální požadavky NÚKIBu [15], bylo rozhodnuto o nevyužití tohoto systému a implementaci řešení využívající AES-256-CBC, na který je aktuálně nahlíženo jako i do budoucna bezpečný algoritmus.

3.2 Grafické rozhraní

Práce s aplikací má probíhat pomocí GUI. Programovací jazyk python nabízí nemalé množství knihoven, které implementaci grafických rozhraní ulehčují [16]. Jelikož je předpoklad funkční podobnosti aplikace s aplikací WinBox, bylo rozhodnuto využít knihovnu DearPyGui [17] [18]. Jedná se o lightweight grafickou knihovnu s hardwarovou akcelerací. Hlavním důvodem výběru této knihovny je možnost jednoduché práce s okny v okně. Předpoklad této funkce je stěžejní pro celou koncepci programu. Stejně tak je nutné, aby grafické rozhraní bylo mutiplatformní.



Obr. 3.2: Grafické rozhraní konfiguračního okna (vývojová verze)

3.3 Pohledy

Spousta systémů síťových prvků nabízí možnost využít tzv. „views“ či „role-based“ přístup [19]. Tato funkcionalita umožňuje definovat, k jakým příkazům a jejich para-

metrům bude mít daný uživatel systému přístup. Právě tato funkcionalita v systému ROS chybí. Díky výše zmíněné volbě architektury (??) využívající lokální databázi dostupných příkazů lze vytvořit pro daného uživatele speciální databázový soubor obsahující pouze příkazy a parametry, které potřebuje ke své práci, a to za účelem zjednodušení či jako „bezpečnostní“ faktor, kdy není uživatel schopen jednoduše procházet či měnit části konfigurace z aplikace, s jejichž fungováním není plně seznámen a/nebo k nim nemá mít přístup.

3.4 Bezpečnost aplikace

Aplikace je navržena s nemalým ohledem na bezpečnost. I přes tento fakt je zde prostor pro zlepšení.

Hlavní oblastí ke zlepšení je oblast zabezpečení databáze. Nejedná se tak o zabezpečení uložených dat, ale spíše slabého místa architektury, a tou je heslo. Na hesla jsou z bezpečnostního hlediska kladeny nároky jako je velká délka či komplexita, tyto požadavky však často nejsou v reálném světě reflaktovány a v případě jejich vynucování může docházet k případům, kdy si uživatelé svá hesla zapisují v čitelné formě elektronicky či na papírky a bloky umístěných poblíž pracoviště či přímo na nich. Hesla jsou dlouhodobě známá jako nejslabší článek jakéhokoli bezpečnostního řetězce, a to především díky tomu, že uživatelé si nastavují hesla krátká a pro ně srozumitelná, jsou tedy náchylná na slovníkové útoky. Právě v krádeži databáze a slovníkovém útoku tkví největší bezpečnostní hrozba tohoto systému.

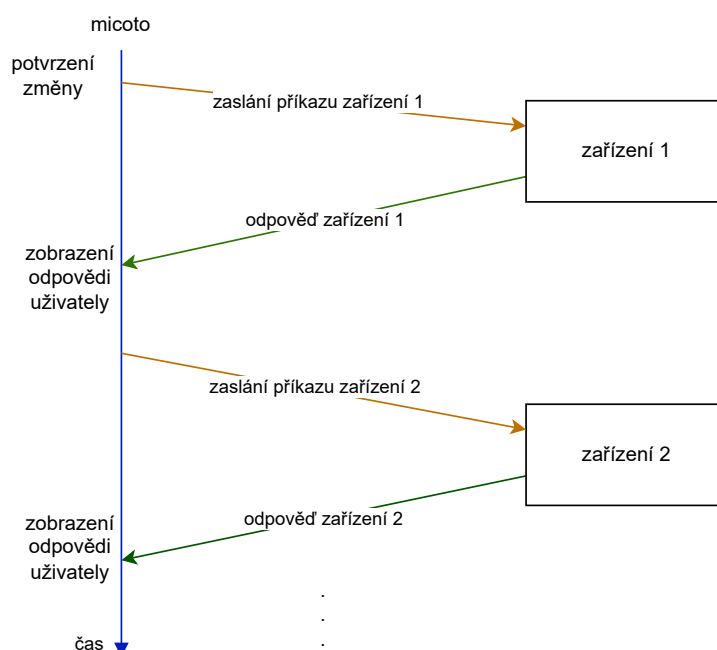
Jako jedno z možných řešení tohoto stavu je odstranění hesel a realizace ověření uživatele pomocí uživatelských certifikátů vydaných důvěryhodnou lokální certifikační autoritou, pro šifrování pak může být použito dlouhé a komplexní heslo, což snižuje riziko při použití slovníkového útoku.

Jako dalším možným řešením je přesun aplikace na server. Při tomto řešení se z aplikace stane centralizované řešení mající své nevýhody spočívající v nutnosti funkčního spojení klient-server a server-konfigurované zařízení. Toto řešení však výrazně snižuje riziko krádeže databáze a zároveň umožňuje využít silný šifrovací klíč a snížit tak riziko prolomení hesla slovníkovým útokem. Zároveň by toto řešení mohlo umožňovat napojení na centrální systém řízení uživatelů ve smyslu Active Directory/LDAP, uložení dat do jedné centrální databáze a vyšším možnostem kontroli nad prováděnými akcemi.

3.5 Hromadné operace

Provádění hromadných operací ve smyslu úpravy konfigurace vícero nařízení zaráz není od výrobce implementována v žádné formě, proto je třeba vymyslet kompletně novou logiku pro řízení této funkcionality.

Pro řízení byl navrhnut kvůli jednoduchosti implementace systém postupného odesílání příkazů jednotlivým zařízením. Po potvrzení konfiguračních změn uživatelem dojde k postupnému aplikování na všech zařízeních, kterých se dané změny týkají. Po příchodu každé odpovědi je uživatel informován a tom, jaký je výsledek požadované operace (odpověď zařízení).



Obr. 3.3: Diagram návrhu realizace hromadné konfigurace

4 Implementace

Aplikace je rozdělena do několika samostatně funkčních celků, a to administrátorské aplikace, samotné konfigurační aplikace a úvodní aplikace, která je otevřena ihned po spuštění aplikace Micoto.

Co se databází týče, je aplikace rozdělena na dvě části. Tyto části jsou databáze s příkazy a databáze s přihlašovacími údaji. Toto rozdělení má své výhody i nevýhody. Pro dané rozdělení databází bylo rozhodnuto kvůli praktičnosti reálného nasazení. Administrátorovi stačí pro změnu dostupných příkazů či změně dostupných zařízení redistribuovat nové databázové soubory s definovanou strukturou, a to nezávisle na ostatních uživateli. Tato skutečnost také umožňuje tyto dvě databáze upravovat nezávisle na sobě a tvořit jejich kombinace.

Aplikace využívá jazyk *Python3*, pro grafické rozhraní knihovnu *dearpygui* a pro práci se databází knihovnu *sqlite3*. Šifrování dat je realizováno pomocí algoritmu AES-256-CBC.

4.1 Komunikace se systémem RouterOS pomocí API

Jak již bylo zmíněno, komunikace probíhá pomocí zabezpečené verze rozhraní *API* na portu 8729. Rozhraní *API* realizuje komunikaci v textové formě. Způsob komunikace bude představen na následujícím příkladu, ve kterém dojde k přihlášení a výpisu IP adres na zařízení.

Jako první krok pro navázání komunikace je otevření spojení, socketu, se zařízením a následné šifrování obsahu pomocí TLS. Ihned po tomto navázání síťového spojení je nutné provést přihlášení.

Výpis 4.1: Ukázka přihlášení při komunikaci s rozhraním API

```
1 <<< /login
2 <<< =name=admin
3 <<< =password=password
4 <<<
5 >>> !done
6 >>>
```

Znaky `<` a `>` v ukázce slouží pouze pro naznačení směru komunikace, v reálné komunikaci se vůbec nevyskytují. Pro zadávání požadavků slouží dva typy slov.

První z těchto slovo typu *příkaz*. Jedná se o klíčová slova systému vykonávající nějakou funkci. Jedná se buďto přímo o dané slovo případně cestu k příkazu, který je dále od kořene stromu. Cesta je určena pomocí lomítkové notace, příklad

/ip/address/print. V jedné větě, sdružení vícero slov různého typu, může být pouze jedno slovo typu *příkaz* a vícero slov typu *atribut*. Příkladem slova typu *příkaz* v ukázce je */login*.

Za slovem typu *příkaz* následuje slovo typu *atribut*. Toto slovo nese informaci o názvu požadovaného parametru a nastavuje jeho hodnotu. Formát tohoto slova je *=nazev_atributu=hodnota_atributu*. Stejně jako dostupné *příkazy*, tak i dostupné *atributy* jsou závislé na aktuálním umístění ve stromu příkazů. Příkladem slov typu *příkaz* v ukázce je *=name=admin* a *=password=password*.

Následující prázdný řádek není náhodný. Systém API pro indikaci ukončení věty využívá prázdný znak. Proto se po každé, ať už přijaté či odeslané, větě v ukázce nachází prázdný řádek.

Následuje odpověď od zařízení *!done*. Znak *!* identifikuje návratový kód příkazu identifikující platnost odeslaného příkazu. Mezi návratové kódy patří *!re* značící bezchybné provedení příkazu, *!done* úspěšné aplikování změn. Nejpodstatnější je však kód *!trap* značící selhání vykonávání příkazu. Za tímto kódem následuje atribut *category* s číselnou hodnotou jež reprezentuje důvod selhání a atribut *message* s bližším textovým popisem chyby.

5 Budoucnost projektu

Ačkoli celý projekt vznikl jako bakalářská práce, mě, jakožto autora, celá práce posunula opravdu hodně kupředu. Důkazem nechť je počet kompletních či částečných přepracování jednotlivých částí kódu a způsoby programování. Jedná se o první autorův projekt takového rozsahu.

Autor si je vědom spousty nedostatků a neoptimalizovaných algoritmů či struktur, a nejen proto se chce autor vývoji aplikace věnovat i po dokončení bakalářského studia.

Závěr

Cílem této bakalářské práce bylo navrhnout a naprogramovat multiplatformní aplikaci v programovacím jazyce Python sloužící k hromadným konfiguracím zařízení s operačním systémem RouterOS prostřednictvím oficiálního zabezpečeného API rozhraní na portu 8792.

V první části se nachází letmé seznámení s výrobcí síťových prvků, systémem RouterOS společnosti MikroTik a způsoby konfigurace tohoto systému. Jedná se především o popis komunikace pomocí zabezpečené verze API rozhraní a pohled na aplikaci WinBox, která byla pro tuto práci inspirací.

Další část již obsahovala informace týkající se vybraných technologií a výběru řešení databáze. Ta využívá knihovnu cryptography, přičemž jsou hesla čifrována pomocí AES-256-CBC. Pro grafický výstup aplikace využívá knihovnu dearpygui. Celá struktura aplikace je navržena tak, aby uživatelé nepřicházeli do styku s hesly v čitelné formě. Dále byla implementována funkcionality "role-based" přístupu či pohledů, díky kterým je možné daným uživatelům zviditelnit pouze tu část konfigurace, kterou skutečně potřebují. Poslední část popisuje práci s aplikací. Ukazuje postup ovládání aplikace uživatelské, administrátorské či konfiguračního okna. Nacházejí se zde také ukázky vybraných částí kódu.

Pro autora to bylo velká zkušenost, jednalo se totiž o jeho první projekt takového rozsahu.

Celý kód je dostupný na platformě GitHub pod licencí MIT – kód, dokumentace, návod.

Literatura

- [1] IEEE. *The world's largest technical professional organization dedicated to advancing technology for the benefit of humanity*. Online. <https://www.ieee.org>. [cit. 2026-03-04].
- [2] Cisco Systems *AI Infrastructure, Secure Networking, and Software Solutions* Online. <https://www.cisco.com/site/us/en/index.html>. [cit. 2026-03-04].
- [3] HP *Laptop Computers, Desktops, Printers, Ink & Toner / HP® Official Site* Online. <https://www.hp.com/us-en/home.html>. [cit. 2026-03-04].
- [4] HP Enterprise *Hewlett Packard Enterprise (HPE) / Czechia* Online. <https://www.hpe.com/cz/en/home.html>. [cit. 2026-03-04].
- [5] Arista Networks *Data-Driven Cloud Networking - Arista* Online. <https://www.arista.com/en/>. [cit. 2026-03-04].
- [6] Ubiquiti Networks *Ubiquiti - Rethinking IT - Ubiquiti* Online. <https://ui.com/>. [cit. 2026-03-04].
- [7] SIA Mikrotīks *MikroTik Routers and Wireless* Online. <https://mikrotik.com/>. [cit. 2026-03-04].
- [8] MikroTik Routers and Wireless *Products* Online. <https://mikrotik.com/products>. [cit. 2026-03-04].
- [9] MikroTik Routers and Wireless *Software* Online. <https://mikrotik.com/download>. [cit. 2026-03-04].
- [10] X.200 : Information technology - Open Systems Interconnection *Basic Reference Model: The basic model* Online. <https://www.itu.int/rec/T-REC-X.200/en/>. [cit. 2026-03-04].
- [11] RouterOS - MikroTik Documentation *Python3 Example* Online. <https://help.mikrotik.com/docs/spaces/ROS/pages/47579209/Python3+Example>. [cit. 2026-03-04].
- [12] SQLite *Home Page* Online. <https://sqlite.org/>. [cit. 2026-03-04].
- [13] sqlite3 — DB-API 2.0 interface for SQLite databases *Python 3.14.0 documentation* Online. <https://docs.python.org/3/library/sqlite3.html>. [cit. 2026-03-04].

- [14] Fernet (symmetric encryption) *Cryptography 47.0.0.dev1 documentation* Online. <https://cryptography.io/en/latest/fernet/>. [cit. 2026-03-04].
- [15] Národní úřad pro kybernetickou a informační bezpečnost *Doporučení v oblasti kryptografických prostředků* Online. https://nukib.gov.cz/download/uredni_deska/Minimalni_pozadavky_v4_FINAL.pdf. [cit. 2026-03-04].
- [16] GuiProgramming - Python Wiki *GUI Programming in Python* Online. <https://wiki.python.org/moin/GuiProgramming>. [cit. 2026-03-04].
- [17] GitHub - hoffstadt/DearPyGui *Dear PyGui: A fast and powerful Graphical User Interface Toolkit for Python with minimal dependencies* Online. <https://github.com/hoffstadt/DearPyGui>. [cit. 2026-03-04].
- [18] Dear PyGui's Documentation *Dear PyGui documentation* Online. <https://dearpygui.readthedocs.io/en/latest/index.html>. [cit. 2026-03-04].
- [19] Cisco Systems *Role-Based CLI Access* Online. https://www.cisco.com/en/US/docs/ios/12_3t/12_3t7/feature/guide/gtclivws.html. [cit. 2026-03-04].
- [20] Zabbix *The enterprise-class open source observability solution* Online. <https://www.zabbix.com>. [cit. 2026-05-25].
- [21] LibreNMS Online. <https://www.librenms.org>. [cit. 2026-05-25].

Seznam symbolů a zkratek

CHR	Cloud Hosted Router
ROS	Router Operating System

Seznam příloh